

B.Tech Project Report: A Unified Framework for Bidirectional English-Hindi Translation

Author: Harsh Dahiya

Institution: Indian Institute of Technology (IIT) Patna

Degree: Bachelor of Technology (B.Tech)

Compute Infrastructure: NSM PARAM Rudra (NVIDIA A100 80GB), Kaggle (Dual T4), Google Colab (T4), Apple Silicon (M2)

1. Abstract

This project evaluates the efficacy, architectural stability, and multimodal integration of modern decoder-only Large Language Models (LLMs) engineered for Indic languages. The research maps a chronological progression from baseline sequence-to-sequence evaluation to the deployment of foundational Indic LLMs (Sarvam AI Sarvam-1, Socket Labs Pragna-1B, and BharatGen Param-1). The core objective was to establish an optimal Parameter-Efficient Fine-Tuning (PEFT) pipeline for bidirectional English-Hindi machine translation.

This report documents the rigorous methodologies applied to select optimal training datasets, resolve the mathematical breakdown of 4-bit quantization on custom architectures, mitigate severe tensor geometry violations, and successfully deploy a real-time, cascaded Speech-to-Speech Translation (S2ST) system utilizing OpenAI's Whisper and Microsoft Azure Neural TTS.

2. Dataset Strategy & Benchmarking

Selecting the appropriate data pipelines was critical to ensuring the LLMs did not simply memorize sentence pairs but genuinely learned linguistic routing.

2.1 Training Corpus: AI4Bharat Samanantar

For the core fine-tuning phase, the project utilized the **Samanantar** dataset. Samanantar was selected because it is the largest publicly available parallel corpus for Indic languages. It aggregates massive amounts of data from news, government documents, and web crawls. This rich, multi-domain contextual data is strictly necessary to properly instruction-tune massive causal LLMs, granting them the deep, diverse vocabulary required for nuanced, real-world Hindi-English translations.

2.2 Evaluation Benchmark: IN22-Gen vs. IIT Bombay

Initial proof-of-concept testing utilized the **IIT Bombay (IIT-B)** dataset. However, for the final

rigorous comparative analysis, the evaluation was migrated to the **AI4Bharat IN22-Gen Benchmark**.

- **The Reason for the Shift:** The IIT-B dataset is older and frequently included in the massive pre-training mixtures of modern LLMs. Using it for evaluation creates a high risk of "data leakage," where models score artificially high because they have already memorized the test data during base pre-training. The IN22-Gen benchmark is the modern industry standard, providing high-quality, human-verified, structurally complex sentences that rigorously test the model's true zero-shot and generalization capabilities without contamination.

3. Phase 1: Baseline Paradigms (Encoder-Decoder vs. Causal)

Before scaling to massive Indic LLMs, early testing was conducted to understand the fundamental behaviors of standard architectures on translation tasks.

- **Causal Language Models (GPT-2):** Initial testing revealed that standard Causal LMs lacking multilingual pre-training suffer from a severe "Embedding Tax." GPT-2's Byte Pair Encoding (BPE) heavily fragmented the Devanagari script.
Results: The model produced valid Devanagari characters (e.g., generating "कृपया नहीं.") but semantically meaningless translations(as the original sentence was entirely different). It rapidly overfitted, with validation loss increasing steadily from **1.97 to 2.48** while training loss dropped.
- **Multilingual Seq2Seq (mT5-small):** Training a generalized 101-language model on a limited 50k sentence dataset proved inefficient. The vast majority of its parameters were dedicated to a massive 250k-token vocabulary, starving the neural capacity needed for translation logic.
Results: It suffered from data starvation, yielding a **SacreBLEU of 0.17** and a **chrF of 8.07**. Validation loss worsened from **4.66 to 5.24**.
- **Domain-Specific Pre-training (Helsinki-NLP / opus-mt-en-hi):** Evaluated as a strict zero-shot baseline.
Results: Using Greedy Search, it achieved a **SacreBLEU of 8.43** and **chrF of 32.90**. Counter-intuitively, forcing Beam Search (num_beams=5) degraded performance (SacreBLEU dropped to **6.84**). Furthermore, it failed idiom tests. This proved that while Seq2Seq models hold strong grammatical baselines, they suffer from severe domain mismatch compared to modern localized models.

4. Phase 2: PEFT Methodology & Architectural Optimization

To achieve production-level translation accuracy, the research pivoted to fine-tuning sovereign Indic foundation models (Sarvam-1 [2B], Pragna-1B [1.25B], and Param-1 [2.9B]).

4.1 The LoRA Optimization Stack

To train multi-billion parameter models within HPC VRAM limits, Parameter-Efficient Fine-Tuning (PEFT) was deployed using Low-Rank Adaptation (LoRA). The optimal configuration applied across all models to ensure a strict 1:1 scientific comparison was:

- **Rank (r) = 64:** A higher rank was chosen specifically for translation tasks. A larger "brain capacity" ensures the adapter can learn and retain the complex syntactic restructuring required when moving from English (SVO) to Hindi (SOV).
- **Alpha = 128:** Scaled proportionally to the rank to ensure gradient magnitudes remained stable during backpropagation.
- **Target Modules = "all-linear":** Instead of just targeting attention heads, the adapters were injected into the Multi-Layer Perceptrons (MLPs)—the "reasoning centers" of the model—maximizing the model's bilingual learning capability.
- **Dropout = 0.05:** A conservative dropout rate of 5% was implemented as a regularization mechanism.
- **Precision = Native BFloat16 (torch.bfloat16):** BFloat16 (Brain Floating Point) was strictly enforced on the A100 cluster. Unlike standard fp16, bfloat16 possesses the same dynamic exponent range as full 32-bit floats. This mathematically prevents underflow/overflow crashes (NaN loss) during massive gradient updates while maintaining half the memory footprint of fp32.

4.2 The 4-bit Quantization Breakdown & The Native BF16 Shift

Initial implementations utilized 4-bit NF4 double quantization (BitsAndBytesConfig) to radically compress the models. While Pragna-1B and Sarvam-1 survived the compression, Param-1 suffered a catastrophic mathematical collapse.

- **The Failure:** During 4-bit training, Param-1 returned 'loss': '0', 'grad_norm': 'nan', 'entropy': 'nan'.
- **The Diagnosis:** The bitsandbytes library relies on highly optimized C++ binaries designed for standard transformer geometries. Because Param-1 utilizes a custom, non-standard architecture (**modeling_parambharatgen.py**), the 4-bit quantization algorithm physically shredded the model's weight matrices during the forward pass, triggering immediate NaN (Not-a-Number) gradient explosions.
- **The Native BF16 Resolution:** To overcome this, 4-bit quantization was completely abandoned. By removing the compression bottleneck and shifting entirely to native bfloat16, the training math stabilized entirely. This decision ensured an apples-to-apples baseline for the final architectural evaluation, proving that the models were being judged on their true structural capability rather than their resistance to quantization noise.

5. Architectural Limitations & Codebase Debugging

During the comparative evaluation phase, severe performance degradation and hardware faults were isolated within the Param-1 architecture. The debugging process required iterating through over-constrained parameters before establishing a true 1:1 baseline.

5.1 The "Over-Optimization" Trap and the 0.74 BLEU Score

Initial attempts to fine-tune Param-1 using 4-bit quantization resulted in catastrophic

mathematical collapse (loss: 0, grad_norm: nan). To force the model to train without exploding, an overly restrictive "straightjacket" configuration was temporarily adopted (*reducing LoRA Rank to $r=16$ and targeting only the q_proj and v_proj attention heads*).

When this heavily restricted model was evaluated—combined with a forced `use_cache=False` override to prevent the custom architecture from crashing during inference—it yielded a catastrophic **BLEU score of 0.74**, indicating complete contextual hallucination.

5.2 The 1:1 Control Test Restoration

To determine if the failure was due to the restricted parameters, 4-bit quantization noise, or the architecture itself, the methodology was corrected to a strict **1:1 Control Test**. 4-bit quantization was completely abandoned. Param-1 was loaded natively in `bfloat16`, and the original, aggressive optimization parameters used for Sarvam-1 and Pragna-1B were restored (*Rank $r=64$, `target_modules="all-linear"`*).

This restored mathematical stability during training. However, transitioning to native inference (`use_cache=True`) exposed critical, hardcoded bugs in the Param-1 architecture:

- **Step Zero Bug:** The custom architecture (`modeling_parambharatgen.py`) blindly attempted to read the shape of the Key-Value cache (`past_key_values[0][0].shape[2]`) during the first step, failing to account for an initially empty (`None`) state. Surgical patches (*if `past_key_values` is not `None`...*) were applied manually to the source code.
- **The Fatal Tensor Geometry Violation:** Bypassing the `NoneType` errors forced the model into tensor geometry calculations, which broke down entirely: **`torch.AcceleratorError: CUDA error: device-side assert triggered (Index out of bounds at apply_rotary_pos_emb:190)`**. The internal mathematics calculating sequence lengths for Rotary Positional Embeddings (RoPE) were fundamentally misaligned for standard Hugging Face text generation.

5.3 Final Remediation and Inference Impact

Param-1's codebase was deemed strictly incompatible with standard PEFT inference pipelines. To extract final metrics, the evaluation script was reverted to the `use_cache=False` override.

Conclusion: Thanks to the restored 1:1 Control parameters (*native `bfloat16`, $r=64$, `all-linear`*), Param-1's translation quality substantially improved from the initial 0.74 to a **BLEU score of 12.19** and a **COMET score of 59.79**. However, disabling the KV cache lobotomized the model's Grouped-Query Attention (GQA), inducing contextual amnesia. Consequently, it still severely lagged behind Sarvam-1, formalizing the empirical proof that raw parameter scale (2.9B) cannot compensate for a rigid, unoptimized custom tensor geometry.

6. HPC & MLOps Optimization

Operating distributed high-performance computing tasks required significant infrastructure management:

- **Multi-GPU Spillage (CUDA OOM):** When loading models sequentially on SLURM-managed environments, default configurations (`device_map="auto"`) attempted to split layers across multiple GPUs. If co-occupied, this caused silent inter-node communication crashes. This was resolved by implementing strict hardware firewalls

(`export CUDA_VISIBLE_DEVICES=0`) and hardcoded tensor mappings (`device_map={"": 0}`).

- **JIT Compilation Constraints:** Missing standard Linux developer headers caused Triton's Just-In-Time compiler to fail. An isolated compiler toolchain (`gcc_linux-64`, `gxx_linux-64`) was built natively into the Conda environment to successfully compile necessary CUDA kernels.
- **Disk Quota Exhaustion:** The SFTTrainer saved 5GB to 15GB optimizer states every 500 steps, crashing training at 98% completion (**[Errno 122] Disk quota exceeded**). Implementing `save_total_limit=2` enforced automated garbage collection, allowing training to resume seamlessly from optimal checkpoints without risking the HPC quota.

7. Multimodal Integration: Speech-to-Speech (S2ST) Pipeline

To prove real-world viability, the optimal LLM (Sarvam-1) was deployed into a live, real-time cascaded Speech-to-Speech Translation prototype utilizing Google Colab's dynamic T4 CUDA allocation.

7.1 API Paywalls & Administrative Bottlenecks

Initial system architectures attempted to utilize external, India-centric APIs for the audio processing bookends. However, this approach presented severe logistical roadblocks:

- **Enterprise Vendor Lock-in (Gnani.ai):** Attempts to utilize Gnani.ai's proprietary ASR and TTS models were abandoned. Their ecosystem operates on a strictly B2B, paywalled commercial structure, offering enterprise demos.
- **Administrative Delays (Bhashini):** The pipeline was subsequently pivoted to Bhashini (Digital India Bhashini Division) to leverage government-backed infrastructure. While the backend Python REST integration (including Base64 audio payload encoding) was successfully engineered, the deployment was halted indefinitely due to protracted administrative delays in the approval of developer API keys on the portal.
- **The Open-Source Resolution:** To guarantee a reliable, zero-cost demonstration for the B.Tech defense, the architecture was aggressively refactored to eliminate third-party API dependencies. The pipeline was shifted to utilize open-weight models (Whisper) loaded directly into GPU VRAM and accessible unauthenticated endpoints (Azure Edge-TTS).

7.2 Strategic Shift to Whisper Large V3 Turbo

Initial prototyping utilized *openai/whisper-small.en* for Automatic Speech Recognition (ASR). Qualitative testing revealed significant drop-offs in accuracy due to sensitivity to Indian English accents, variable speaking pacing, and background noise.

- **The Upgrade:** The pipeline was migrated to *openai/whisper-large-v3-turbo*. This state-of-the-art model retains the massive linguistic reasoning capabilities of the 1.5B parameter V3 architecture—handling heavy accents and phonetic code-switching flawlessly—while being mathematically pruned for real-time latency. Loaded in float16, it

comfortably shared the 16GB T4 VRAM alongside Sarvam-1.

7.3 Strategic Shift to MS Azure Neural TTS

Initial text-to-speech output relied on *Google TTS (gTTS)*.

- **The Limitation:** gTTS generates a robotic, emotionless, default female voice lacking professional acoustic qualities.
- **The Upgrade:** The pipeline was transitioned to the edge-tts library, tapping directly into *Microsoft Azure's Neural TTS API*. This allowed access to highly realistic, gender-specific, and localized Indian dialects (e.g., hi-IN-MadhurNeural and en-IN-PrabhatNeural). The neural engine introduced natural breathing pauses and accurate Hindi phonetics, fundamentally elevating the demonstration grade of the final S2ST system.

7.4 Local Hardware Constraints (MPS) vs. CUDA Deployment

- **Apple Silicon (MacBook Air M2) NDArray Limits:** Attempting local MPS inference on macOS crashed due to Apple's 4GB single-tensor limit (total bytes of NDArray > 2^{32}) when pre-allocating an 8,192 token attention matrix. To ensure a flawless live demonstration, the pipeline was successfully migrated to a Google Colab (T4) environment leveraging dynamic CUDA memory allocation.

8. Quantitative & Qualitative Results

Evaluations were performed natively in bfloat16 precision against 1,000 unseen sentences from the **IN22-Gen Benchmark**. The primary metric utilized was **COMET**, providing superior neural semantic scoring over strict n-gram matching (BLEU/chrF). The SOTA Encoder-Decoder model (IndicTrans2) is included as a reference upper-bound oracle.

8.1 Translation Quality Metrics

English to Hindi Translation (IN22-Gen Benchmark)

Model Architecture	Parameters	BLEU Score	chrF Score	COMET Score
IndicTrans2 (SOTA Oracle)	~1.0B	23.40	53.80	82.10
Param-1 (Control)	2.9B	12.19	42.94	59.79
Pragna-1B	1.25B	8.64	32.39	63.62
Sarvam-1	2.0B	19.92	47.67	77.02

Hindi to English Translation (IN22-Gen Benchmark)

Model Architecture	Parameters	BLEU Score	chrF Score	COMET Score
IndicTrans2 (SOTA Oracle)	~1.0B	29.10	57.50	85.40
Param-1 (Control)	2.9B	7.32	19.12	38.19
Pragna-1B	1.25B	18.86	46.10	81.09
Sarvam-1	2.0B	24.37	54.51	86.02

8.2 Qualitative Observations & Hallucination Analysis

- **Sarvam-1 Semantic Mastery:** Achieved exceptional alignment (86.02 COMET in Hi->En). It accurately translated complex contexts and specialized vocabulary, scoring within striking distance of the dedicated IndicTrans2 translation engine despite being a general-purpose causal model.
- **Pragna-1B Efficiency:** Operated flawlessly as a lightweight baseline. While grammatically sound, it exhibited magnitude flaws due to smaller vocabulary depth (e.g., translating "thousands" to "lakhs" in example sentences attached with report), resulting in lower BLEU scores despite high COMET readability.
- **Param-1 Catastrophic Forgetting:** Stripping the KV cache resulted in complete context collapse. When processing clinical text in the Hindi-to-English task, the model hallucinated random training data artifacts, outputting phrases such as: *"Police reached the spot and brought the situation under control."*

9. Conclusion

This research provides definitive proof that raw parameter scale does not equate to performance in Parameter-Efficient Fine-Tuning. **Param-1 (2.9B)**, despite possessing the largest parameter count, suffered a catastrophic performance collapse. The rigorous 1:1 native BFloat16 control test definitively isolated this failure not to domain overfitting or quantization noise, but to a fundamentally unoptimized, custom KV cache geometry that fractured during standard Hugging Face generation workflows.

Conversely, **Sarvam-1 (2B)** demonstrated exceptional architectural stability and semantic alignment. Its native compatibility with standard attention mechanisms, combined with highly optimized token fertility for Indic scripts, proved vastly superior for PEFT applications. By

integrating this structurally resilient LLM within a GPU-accelerated Whisper Turbo ASR and Azure Neural TTS pipeline, this project delivers a highly viable, real-time, and scalable solution for cross-lingual speech-to-speech translation in Indian languages.